

MQ-SMF-CSV

A basic formatter for IBM MQ's z/OS SMF records that can run on Windows and Unix systems, simplifying the import of statistics to spreadsheets, databases and analytics solutions.

The program can now also format MQ AMS records (type 180) and z/OS Connect EE audit records (type 123).

Description

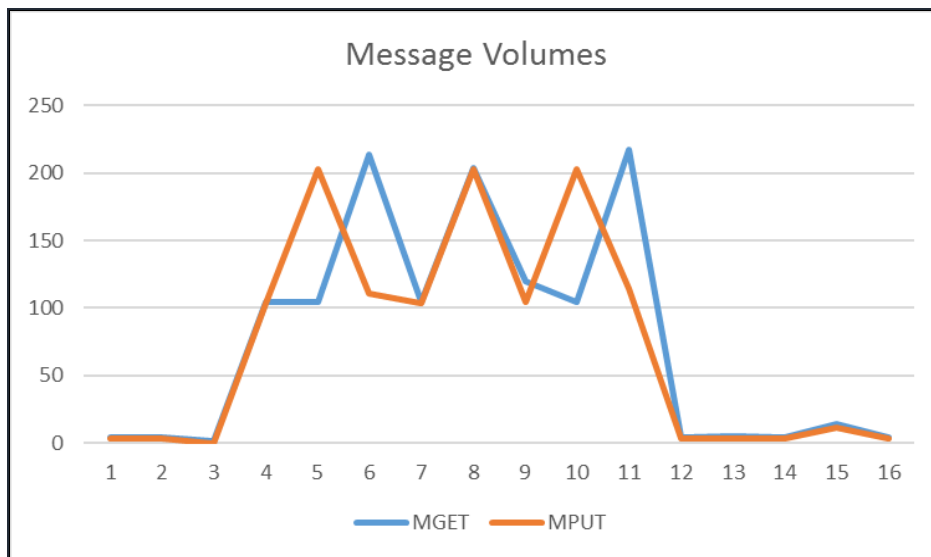
On z/OS, IBM MQ generates SMF accounting and statistics records to track the usage of the product. These are often used for capacity planning, for performance tuning, or for chargeback.

This package has been created to simplify the job of doing your own analysis of MQ SMF records from a z/OS system, by making it easy to import formatted records into a database or spreadsheet. The program runs on a Windows or Unix system, taking input data from a z/OS file, and creating standard CSV (comma-separated value) output files. A new option also allows generation of JSON-formatted output, but I have not renamed the program.

Unlike packages such as **SupportPac MP1B**, there is no interpretation at all of the SMF data. That can be done by your own spreadsheet or database programs and macros. And unlike the sample program **csq4smfd** from the MQ product, the output is intended to be used by other programs rather than being directly human-readable.

This makes the code for the program very easy to understand; all the structures in the SMF records are printed directly.

Importing the data into a spreadsheet makes it easy to construct graphs like this:



History

See CHANGES.md in the source directory.

Acknowledgements

I would like to acknowledge the involvement of Lyn Elkins who spurred the initial development of this tool, provided sample data and hands-on testing, and suggested various enhancements.

Also thanks to Doug Burns of the MQ Service team for the **cpvbus** conversion tool and Andrew Mattingly of IBM Australia for the AMS record formatter.

Using the mqsmfcsv program

Collect the MQ SMF data in the usual way. See

http://www.ibm.com/support/knowledgecenter/SSFKSJ_9.1.0/com.ibm.mq.mon.doc/q038210_.htm?lang=en for an introduction to collecting MQ's SMF data.

After initial collection, the SMF records must be dumped to a dataset using the IFASMFDP program. Use this to select just the MQ 115 and 116 records and reduce the total size of data to be processed. See https://www.ibm.com/support/knowledgecenter/SSLTBW_2.1.0/com.ibm.zos.v2r1.ieag200/dump.htm for more information on this program.

This JCL shows how to run the IFASMFDP program and select only the MQ records:

```
//METDP JOB (????,????), 'DUMP MQ SMF', NOTIFY=?????????
//SMFDMPA2 EXEC PGM=IFASMFDP
//DUMPIN DD DISP=SHR, DSN=MPXCAT.SMF.MPX1.MANZ
//DUMPOUT DD DSN=MET.SMF.DATA,
// DISP=(NEW,CATLG,DELETE),
//* DATACLAS=COMT, RECFM=VB,
// RECFM=VB,
// VOL=SER=Q70006,
//* SPACE=(CYL,(5,2),RLSE), UNIT=SYSDA
// SPACE=(CYL,(100,200),RLSE), UNIT=SYSDA
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
INDD(DUMPIN, OPTIONS(DUMP))
OUTDD(DUMPOUT, TYPE(115,116))
/*
//
```

Downloading z/OS datasets for processing

There are two options for the file format as it is copied from the z/OS system. The default, and the original mechanism supported in this program, uses the RDW option.

For sites where the ftp command may not be available (a lot of sites now have security restrictions, and enforce alternatives such as sftp, which does not seem to support the same options), then it is now possible to use the raw, non-RDW format. You may additionally need to convert the SMF dataset first to an unstructured format before these other transfer programs can be used. Use of the **cpvbuss** program is discussed below.

Download with RDW option

Files on z/OS are usually not simply byte streams but have internal structure such as having known and variable record length. That means that the file cannot be directly read by standard POSIX file I/O functions. Instead, the file must be converted before it can be read. This conversion can be done by the ftp server on z/OS when transferring files to Windows or Unix. It is not done by default, and a special option must be selected to enable that processing. Other file transfer programs may also work, but this one is known to succeed.

To download the SMF data, run the ftp client on your workstation and connect it to the z/OS system. Once you are logged in, use the command "QUOTE SITE RDW" or "LITERAL SITE RDW". Also make sure you are transferring files in binary format. Then download the dataset containing your SMF data.

```
$ ftp winmvs41
Connected to 9.20.1.1
220-FTPSERVE IBM FTP CS V2R2 at WINMVS41, 12:00:00 on 2016-03-10
220 Connection will close if idle for more than 30 minutes
User (winmvs41:(none)): met
331 Send password please.
Password:
230 MET is logged on. Working directory is "MET.".
ftp> BINARY
200 Representation type is Image
ftp> QUOTE SITE RDW
200 SITE command was accepted
ftp> GET 'MET.SMF.DATA' /tmp/smfdata.bin
200 Port request OK
125 Sending data set MET.SMF.DATA
250 Transfer completed successfully
ftp: 792532 bytes received in 0.30 Seconds 2641.77Kbytes/sec.
ftp> quit
```

Download without RDW option

Getting the file from z/OS to the workstation is the same process as above, except that the “QUOTE SITE RDW” is not needed.

Instead, you now need to add the “-f NORDW” option to the invocation of mqsmfcsv.

If the wrong format is specified, then there will likely be “unknown” record types and potentially corrupted output or the program crashing (though of course the input file is not touched so you can have another go with the correct option).

Converting the dataset from VBS before download

SMF data is usually written to a VBS dataset on z/OS. The ftp command can process that format in its transfer, but not all file transfer tools can. If the tool you need to use (for example **sftp**) needs an alternative format, then the **cpvbuss** program provided in this package should help. Both the C source code and binary are supplied.

First, upload the program binary to z/OS (or take the source and compile it yourself there). You may also need to set the program to be executable.

Then run it with two parameters - the name of the VBS dataset to be read, and the name of a USS file to be written.

The newly-created USS file can then be transferred to the workstation as already described, and processed with the "nordw" option.

Running the program

The output files are written to a directory that you name on the command line. By default, this is the current directory. One file is created or appended for each type of element - QPST, WTID etc. The files normally contain a line of column headers, followed by the actual data with each field separated by a comma. Spreadsheets such as LibreOffice or Excel can then import the data directly; importing to Access has also been tested.

No options are required for the command; default options are usually adequate. Just give the name of the input file using the **-i** parameter. The simplest operation is

```
mqsmsfcsv -i /tmp/smfdata.bin
```

Command Options

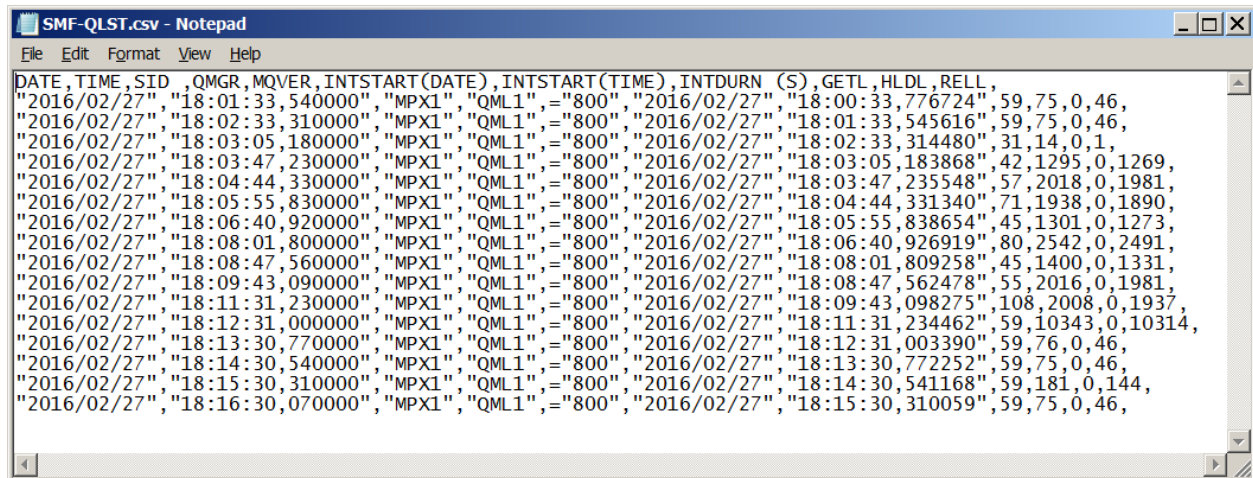
The options to the mqsmsfcsv command are:

- **-?** Print the help text for the program.
- **-a** Append to existing files if they exist. Default is to overwrite files.
- **-i** Input file. If not given, *stdin* is assumed. Piping via stdin requires "rdw" format input files.
- **-o** Directory for the output files. Use "-" to specify stdout; this would probably be useful only for streaming JSON output.
- **-h yes | no** Print column headers for new output files. Default is "yes". You may not need a set of column headings when the structure is already known. The headings are never added a second time when appending to an existing output file.
- **-f rdw | nordw** How was the file downloaded from z/OS. Were the Record Descriptor Words included in the copy.
- **-f sql | json | csv** Create output in one of these formats. The input and output format specifications can be combined in a single option with a comma separator. For example "-f rdw,sql".
- **-f oldcolsonly** See the "multiple version support" section below
- **-p <file>** Copy SQL statements from this file to the start of the generated DDL file. If this option is not used, and the DB2 DDL format is selected or left as default, then some default statements are given to manage the PAGESIZE of a bufferpool and tablespace based on DB2 requirements. If the filename is "-" then these default statements are omitted. Otherwise the contents of specified file is copied to the DDL output.
- **-e <file>** Copy SQL statements from this file to the end of the generated DDL file. This allows you to have a template of commands executed after tables have been created, for example to add INDEX options.
- **-b <db2|mysql>** Default is "db2". Determines the format of the generated DDL.
- **-r** The program by default puts "≡" in front of numeric-appearing strings to assist the automatic importers in common spreadsheets with recognising these are indeed strings. Use this flag if you do not need this.
- **-m** Maximum number of records to process. This may be useful when you only want to work with the first few records in the file; perhaps to test the process before spending time formatting millions of records.
- **-s (deprecated - use -f sql option instead)** SQL mode to generate simple DDL statements to create tables with the correct column names and types and assist with importing to a SQL database
- **-t** Ticker. Print a progress statement during processing every T records. Default is 10000.
- **-c** Restart the formatter from a checkpoint. This may help if you have run out of disk space part-way through a long run, formatting a very large file. A record of current progress is made at each ticker interval, allowing restarts to continue from that point.
- **-y <AMS record type>** The AMS component of MQ can have a configurable record type. The default is **180**, but this option is required if your system has been set up to use a different type.
- **-d** Debug level. Set to 1 or 2 to dump binary copies of the records during processing.

Output formatting

Little attempt is made here or in the code to explain the column headers; these mostly come straight from the field names and descriptions in the structures. More information can be found in the *csqdsmf.c.h* header file on z/OS or the product documentation.

Here is an example output file generated from some of the sample data:



```
DATE, TIME, STD, QMGR, MOVER, INTSTART (DATE), INTSTART (TIME), INTDURN (S), GETL, HLDL, RELL,
"2016/02/27", "18:01:33,540000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:00:33,776724", 59, 75, 0, 46,
"2016/02/27", "18:02:33,310000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:01:33,545616", 59, 75, 0, 46,
"2016/02/27", "18:03:05,180000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:02:33,314480", 31, 14, 0, 1,
"2016/02/27", "18:03:47,230000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:03:05,183868", 42, 1295, 0, 1269,
"2016/02/27", "18:04:44,330000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:03:47,235548", 57, 2018, 0, 1981,
"2016/02/27", "18:05:55,830000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:04:44,331340", 71, 1938, 0, 1890,
"2016/02/27", "18:06:40,920000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:05:55,838654", 45, 1301, 0, 1273,
"2016/02/27", "18:08:01,800000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:06:40,926919", 80, 2542, 0, 2491,
"2016/02/27", "18:08:47,560000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:08:01,809258", 45, 1400, 0, 1331,
"2016/02/27", "18:09:43,090000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:08:47,562478", 55, 2016, 0, 1981,
"2016/02/27", "18:11:31,230000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:09:43,098275", 108, 2008, 0, 1937,
"2016/02/27", "18:12:31,000000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:11:31,234462", 59, 10343, 0, 10314,
"2016/02/27", "18:13:30,770000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:12:31,003390", 59, 76, 0, 46,
"2016/02/27", "18:14:30,540000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:13:30,772252", 59, 75, 0, 46,
"2016/02/27", "18:15:30,310000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:14:30,541168", 59, 181, 0, 144,
"2016/02/27", "18:16:30,070000", "MPX1", "QML1", "=", "800", "2016/02/27", "18:15:30,310059", 59, 75, 0, 46,
```

For this particular set of QLST elements, the qlstgetl and qlsthldl fields are then properly described in the MQ KnowledgeCenter or in the csqdsfmfc.h header file.

Note that if the data contains SMF records from non-MQ sources, these are copied to any debug output file, but otherwise ignored.

Strings

Strings are put into the CSV file surrounded by quotes. When a string looks as if it could be a hex or decimal numeric value (for example a correlation ID), the default behaviour of the program is to also add “=” before the value. This helps avoid the problem where spreadsheets try to interpret the value as a number and get it wrong. If you do not need the “=”, for example when importing to a database that already knows how to process the field as a string regardless of its content, then use the **-r** flag to **mqsmfcsv**.

Array indices are surrounded by “{}” characters. This is because Access does not accept the more usual “[]” indicators.

Date and time values

Date and time are split into separate columns; the time includes a microsecond portion which is separated using ‘.’ as using ‘.’ as the decimal point seems to confuse the automatic importers on several spreadsheets. There is no information directly available about timezone offsets, so it is possible that the timestamps could be either GMT (CUT) or local time, depending on system configurations. I have seen a couple of places where timestamps are nonsensical; this seems to happen for a couple of records where other fields indicate no work was done (my guess is the MQ product code does not blank out an otherwise-unused record). And there are also timestamps where the STCK time is zero; these are converted to blanks in the output. Again, these seem to come in records where no real activity has taken place.

Durations in CSV files are split into separate columns for seconds and microseconds. This is shown in the column headings.

The JSON output for times uses the more natural “.” for the microsecond delimiter.

Multiple Version Support

Newer versions of MQ may introduce additional fields to some record subtypes. If the input data to this program contains SMF records from queue managers which are at different levels, this means that you might get different sets of fields in the formatted version. That could cause confusion when importing the data to a database or to a spreadsheet – there would be different numbers of columns in the CSV file. This became particularly obvious with

MQ 10, where the QSG name was added to all records, and it appears near the beginning of the columns, not at the end.

The original solution to this was to give an error message, and to exit the formatter. You would need to split the input data by queue manager (on z/OS) before downloading it and formatting.

The solution now is to format older data as if it were coming from queue managers at the latest known level. Columns are printed for all known fields.

Fields which were not part of the older records are printed with dummy values, enough to recognise that these are not real values. Strings are printed with '.'; integers are set to -1 (which is particularly obvious) and dates are set to 1970.

The MQ 10 QSG will be given as a blank string if the qmgr is not a member of a QSG, whereas data from a pre-V10 qmgr will be shown as "...". This should help distinguish between the situations. As well, of course, as having the MQ version in the printed record. Note that some record subtypes already included a QSG Name; this will be duplicated in the common core block of fields.

SQL queries may now need to be sensitive to these special values. For example, it may not be appropriate to produce a total for all numbers in a column; you may need to exclude rows where the value is -1.

If you want to have only the real columns AND you know that the data only comes from queue managers at the same level, then you can use the **"-f oldcolsonly"** option to the command.

Working with the SQL mode

When you use the **"-s"** option to the command, a file containing DDL statements is generated. This is to help with importing into databases such as DB2 which need to know the shape of the data before importing it, even from a CSV file. Using this option disables printing of column headers within the CSV file itself. The DDL is trivial, dropping and recreating tables into which you can import the data. When running in this mode, durations are printed as a single column combining both seconds and microseconds.

This example shows how to import some data into a DB2 database from a Command Line Processor window.

```
db2 connect to tiny
db2 -tvf c:\temp\smf\smf-mqtables.ddl
db2 load client from c:\temp\smf\smf-gest.csv of del insert into mqsmf.gest
db2 load client from c:\temp\smf\smf-wq.csv of del insert into mqsmf.wq
```

It is recommended that you run this as a script that sends output and errors to a log file, perhaps using **tee** or similar. That makes it easier to review later for errors in the formatting.

Using DB2 - Tablespaces

For DB2, you may also now need to create a new tablespace with a page size of at least 8K that can be used during the table creation process.

On some DB2 installations, the default tablespaces have page size of 4K; at least one table (WQ) is now large enough that it seems to require the larger page size and an error occurs when the table is created. Once the additional table space has been created, then it seems to be selected automatically, so that no further or regular changes are needed to the DDL. The default behaviour of the program is to put these commands in the generated DDL. That can be suppressed via the **-p** command line option. If the parameter is **"-"** then nothing extra is written; if the parameter is a file, then its contents are copied unchanged to the generated DDL file.

```
create bufferpool mqp8k immediate all dbpartitionnums size 1000 automatic
pagesize 8192;
create tablespace mqp8k pagesize 8192 bufferpool mqp8k;
```

Using MySQL

This program has also been tested with importing the data to a MySQL database.

A Linux script (*testing/load-mysql.sh*) shows how to connect to a local MySQL database, execute the DDL and then import the data.

Note that the default configuration for MySQL does not permit loading of data from local files. For simplicity, you can add the line "local-infile=true" to the database configuration file (default on Linux is */etc/my.cnf*), and restart the database. A production environment may have alternative mechanisms for managing the imports.

Working with the JSON Mode

Output from the formatter can be written as JSON structures to a single file or streamed to stdout. Each structure contains the same information as the SQL or CSV rows, with the record type, subtype and structure name included. This may be interesting to people wanting to feed the data to JSON analytics programs.

This example shows the start of a JSON formatted SMF record:

```
{
  "recordType" : 116,
  "recordSubType" : 0,
  "structure" : "QMAC",
  "date" : "2015/11/23",
  "time" : "11:00:00.020000",
  "lpar" : "H019",
  "qmgr" : "MQPC",
  "mqVersion" : "800",
  "authorisationId" : "IMS      ",
  "correlId" : "F0F2F3F6C2C3F1E4C4D6C340",
  "connectionName" : "PRDC      ",
  "operatorId" : "PLN1231 ",
  "applicationType" : "IMS MPP/BMP",
  "accountingToken" : "0000000000000000000000000000000000000000000000000000000000000000",
  "networkId" : "D7D9C4C340404040044E0A0800000001",
  ...
}
```

Arrays are created as such in the JSON rather than the independent variables used for SQL and CSV output.

For example,

```
...
"prepareCount" : 1,
"latchWaitTimeUS" : [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
"latchWaitCount" : [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
...
```

Sending the output from the formatter to stdout ("-o -") instead of a file allows a streaming-style interaction with other tools that consume JSON.

Design and Development

Binaries are provided for some platforms. For other platforms, you will need to compile the code yourself. Familiarity with Makefiles and compiling C programs is assumed. This section notes a few aspects of the code design and build processes that may help you.

The platforms with pre-compiled code in the release are

- Windows 10
- Fedora 31
- AIX 7.1

Using the provided Makefiles as a template, set the appropriate compilation options for your platform.

For Unix platforms, the "M" script may be a convenient way to override options in the Makefile, as most Unix platforms are likely to be very similar in how to build programs. On Windows, the **m2017.bat** file sets up a basic environment for compiling the code assuming you have Visual Studio 2017. The final section of this document shows how you can use the Cygwin package to compile on Windows.

This program was originally developed based on the MQ V8 SMF structures, and then updated to optionally use the MQ V9 SMF enhancements if you use that level of header file. It will run against data generated by older queue managers but will not compile if an older version of *csqdsmfc.h* is used.

The program is now safe to compile in either 32 or 64-bit mode. Where necessary, variables are declared with a system-independent datatype. The Makefiles run, by default, a **sizeTest** target. This compares the sizes of SMF record structures with known good values, to ensure that offsets are being correctly handled regardless of the compiler being used.

SMF structure definitions

The source code includes a modified version of the MQ product-provided header file on z/OS that defines the structures for SMF elements. This is called *csqdsmfc.h* and can be found on your z/OS installation. That header file is transformed into *mqsmsfstruc*.h*, one for Windows and one for Unix, which are included in the source code here and which you should not try to modify. The transformation is done by the convH program (source code in convH64.c) – it makes sure that standards-compliant datatypes such as *uint32_t* are used that guarantee variables are the correct size regardless of the compiler you are using. The filter program also has code to deal with bitfields in structures – the z/OS syntax in the product header is not recognised by all compilers (or causes offsets to be wrong), so the filter knows how to rewrite the problematic areas.

If a version of MQ is released with a new version of the SMF structures, then you may need to regenerate the transformed file. Download a copy of *cmqdsmfc.h* file to your system. Redistribution and licensing restrictions make it a problem to include that header file in this package directly. Make sure you use a file transfer mechanism that correctly preserves characters from the EBCDIC original. In particular, I have seen times where the square brackets are not handled properly; the ftp command running on the Distributed platform connecting to z/OS to GET the file seems to work well. The name of the header file should be changed to include the version of MQ, for example *csqdsmfc-914.h*. Then run the Makefile for your platform with the **mqsmsfstruc.h** target.

File I/O

The z/OS variable block dataset format generated by the SMF dump program can be processed directly by I/O routines in the z/OS system libraries. They make it easy to just “read the next record” no matter how long it is. But those I/O routines are not part of the normal libraries on other platforms. Using the ftp commands given earlier, the record lengths are embedded into the downloaded file, and directly accessible in the RDW (Record Descriptor Word).

When using POSIX I/O functions such as `fread()`, the program uses the RDW to find out how much more data needs to be read. There are some additional complexities around segmentation, as logical SMF records may be split across several physical file records, but the code handles those, as explained in the comments.

When there is no RDW available, the code takes a rather unsubtle approach of always trying to read the maximum length of a record (32K) and then resetting the pointer for the next read after the headers have been parsed.

SMF Record layout

The standard approach for SMF records, regardless of product that generates them, consists of a header followed by a set of triplets; each triplet containing an offset, length, and number of repeats for an entry. This program deals generically as far as possible with these triplets, but then knows which of the triplets corresponds to which data element for MQ. That information is documented in the MQ KnowledgeCenter at http://www.ibm.com/support/knowledgecenter/SSFKSJ_9.1.0/com.ibm.mq.mon.doc/q038240_.htm?lang=en (for SMF 115 records) and http://www.ibm.com/support/knowledgecenter/SSFKSJ_9.1.0/com.ibm.mq.mon.doc/q038410_.htm?lang=en (for the SMF 116 records).

Formatters

Each data structure is formatted by its own function, contained in the `print*.c` modules. Most of these are very straightforward to understand. Some common macros defined in *mqsmf.h* make it easy to add each field from the structure; they also simplify handling common fields that need to be printed on every type of SMF element.

A few special cases exist, such as handling bitfields or arrays. But again, the code where these are used ought to be obvious and can be used as strategies for handling new types of SMF element.

Sample data

The package includes a tiny amount of sample data in the *testing* subdirectory that can be used to demonstrate the formatting.

Possible enhancements

There are possible enhancements and extensions to this package:

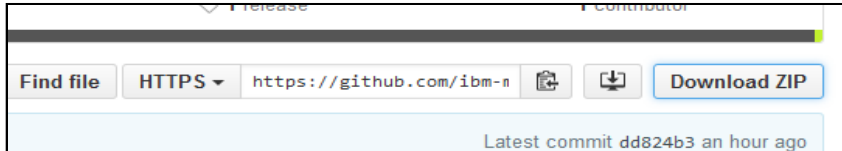
- Obviously newer versions of MQ may introduce new SMF elements that need to be decoded.
- And it would also be interesting to add sample spreadsheets with macros that do some analysis of the SMF data.
- Could add processing for other record types, not just MQ 115 and 116 SMF.

Getting started as a new developer on Windows

For people who may not do much development on Windows, or who do not have a C compiler installed, or do not really know much about Git, here are some steps to get started using a free development environment. It will allow you to modify and rebuild the formatter for Windows.

This section does not attempt to show use of Git to clone or submit source changes back to the master; that's a more advanced topic.

- Go to <http://github.com/ibm-messaging/mq-smf-csv> and download the zip file containing this package



- Unzip the file into a directory of your choice.
- Get your copy of *csqdsmfc.h* from your MQ on z/OS installation into the *src* subdirectory with the rest of the source code. Make sure that the file is correctly transferred, as discussed earlier in this document. Change the name of the file to include the version number, eg *csqdsmfc-903.h*.
- Go to <http://cygwin.com> and download the 32-bit installation of this package (setup-x86.exe). Get the 32-bit version even when your Windows is running a 64-bit OS. This is because there can be problems getting the right compiler options running the 64-bit Cygwin package. It is possible to install both 32 and 64-bit Cygwin packages, but not required for this activity.
- Run the downloaded **setup-x86.exe**. Take defaults for almost everything except where it asks which packages to install. When asked for which mirror site to access, I just picked the first on the list. If you don't know any better, then you should do the same. You need to add the *mingw64-i686-gcc-core* and the *make* packages (both from the "devel" category) to the already selected set by clicking on the "skip" button to change it; clicking "next" will probably add some more packages as prerequisites. Continue through the installation process.
- Once the Cygwin components have been installed there will be a desktop icon for it. Start the terminal session from that icon. You are now in a Unix-like shell with typical Unix commands like **ls** and **cd** available to you.
- Use the **cd** command to switch to the *src* subdirectory beneath where the code was unpacked. Remember that because this is a Unix-like environment, directory separators are "/" and not "\".
- Run **make -f Makefile.gcc.win** to build the program. This builds a standalone copy of the *mqsmfcs.exe* program that can be used on machines without the Cygwin package installed.
- Once you've done this successfully on the downloaded source code, you can start to edit it and rebuild with your own changes.